X-Wing on Cortex-M85: Architecture-Aware Integration and End-to-End Evaluation

Anonymous Author(s)

Anonymous submission

Abstract. X-Wing combines ML-KEM-768, a post-quantum key encapsulation mechanism (KEM), with X25519, a classical elliptic-curve key-agreement scheme. Optimized implementations exist for both components, but improving individual functions does not necessarily improve the complete X-Wing API on a microcontroller with limited issue width and register resources. This paper preserves the cryptographic specification and API of X-Wing draft-10 while jointly adapting the bottlenecks of both computational regimes to the execution resources of the Cortex-M85.

The final integrated implementation comprises eight optimizations. For ML-KEM, we use the 128-bit M-profile Vector Extension (MVE) for the Number Theoretic Transform (NTT), match the NTT storage order to downstream operations, and vectorize 11 data-format transformations. For X25519, we implement a width-3 signed comb for basepoint 9 with a 33,024-byte precomputation table, reschedule field multiplication and squaring, and implement the conditional swap with MVE. We also replace two field inversions that coexist during X-Wing encapsulation with one zero-safe batch inversion and exploit alignment information in common data-copy and initialization routines.

Each candidate was evaluated first in isolation and then in the actual X-Wing API before integration. The reference implementation and the final integrated implementation were alternated within one executable. We performed five independent experiments, reflashing the same board image before each experiment. Across ten paired comparisons, the minimum observed cycle reductions were 19.52% for key generation, 19.75% for encapsulation, 11.66% for warm decapsulation with reuse of an expanded key, and 15.48% for cold decapsulation including key expansion from a 32-byte seed. Both implementations passed standard test vectors, byte-for-byte equivalence checks, invalid-ciphertext and low-order-point tests, and stack-boundary checks. The contribution is not a new cryptographic primitive. It is the selection of where existing techniques apply within X-Wing, their Cortex-M85-specific implementation, and an end-to-end demonstration that function-level gains persist at the complete API level.

Keywords: X-Wing · hybrid KEM · ML-KEM · X25519 · Cortex-M85 · M-profile Vector Extension · embedded cryptographic optimization

1 Introduction

During the transition to post-quantum cryptography, hybrid key establishment is important because it combines classical and post-quantum cryptography instead of relying exclusively on one new algorithm. X-Wing is a hybrid KEM that combines the two shared secrets obtained from X25519 and ML-KEM-768 with SHA3-256 [4,6]. The construction is designed so that one component continues to provide security if the other becomes vulnerable.

On embedded devices, however, optimizations of ML-KEM and X25519 must be evaluated separately to determine whether they translate into performance gains for the complete X-Wing API. We target the Cortex-M85 to evaluate this translation on a high-performance M-profile microcontroller equipped with the Arm M-profile Vector Extension (MVE, also known as Helium). ML-KEM exposes substantial data parallelism that can use 128-bit MVE, whereas X25519 field arithmetic contains long scalar dependency chains. The platform is therefore well suited to studying the integration effects that arise when vector-oriented and scalar-oriented computations share the same execution resources. Limited instruction issue width, a small scalar register file, and finite load/store bandwidth nevertheless prevent the assumption that a faster function will necessarily accelerate the complete X-Wing implementation.

1.1 Related Work

The original X-Wing work presented the construction and its security rationale and evaluated the performance of optimized implementations [4]. To retain implementation simplicity, that work treated both components as black boxes, while also discussing further optimization opportunities across internal ML-KEM processing and the X-Wing key-derivation stage. This paper targets the subsequently specified draft-10 algorithm and API [6].

For ML-KEM implementations, pqm4 provides optimized Cortex-M4 implementations and a common evaluation framework [10]. Prior work reduced the cost of major operations through assembly NTTs, modular arithmetic, and lower memory usage. Work on Helium NTTs presented MVE implementations of the NTT and polynomial multiplication on Cortex-M55 [5]. pqmx extends this line of MVE implementations [14], while SLOTHY uses constraint solving to reschedule cryptographic assembly for a target microarchitecture [1,2].

For X25519, prior studies have developed constant-time Montgomery ladders and optimized field multiplication and squaring for small microcontrollers and the Cortex-M family [11,12,8,7]. A comb using a precomputation table when the input point is fixed to basepoint 9 and a batch inversion that replaces several denominator inversions with one inversion are also established elliptic-curve implementation techniques [9].

This work does not introduce these algorithms, but it also goes beyond simply linking existing code. We connect an MVE NTT to the coefficient representation and downstream storage order of the existing ML-KEM implementation, vectorize 11 data-format transformations and the X25519 conditional swap, imple-

ment and reschedule a fixed-base comb and field arithmetic for the Cortex-M85, and apply a zero-safe batch inversion where two X25519 outputs coexist during X-Wing encapsulation. The contribution boundary is the X-Wing-specific placement of these techniques, their Cortex-M85-specific implementation, and end-to-end validation of the complete API on the same board.

1.2 Contributions

This paper makes the following contributions.

1. We connect an MVE NTT from the pqmx implementation lineage to the coefficient representation and downstream storage order of the existing ML-KEM implementation, eliminate a separate scalar reordering pass, and implement 11 ML-KEM data-format transformations with MVE instructions.
2. We implement a constant-time width-$w = 3$ signed comb for the scalar multiplication whose input is basepoint 9 in X-Wing. We also restructure the stack usage and instruction schedules of X25519 field multiplication and squaring for Cortex-M85, and optimize copies and initialization of internal buffers whose sizes and alignments are known.
3. We replace the scalar xor-mask conditional swap with four groups of 128-bit MVE operations. During encapsulation, we combine the inversions of two simultaneously live X25519 outputs and preserve the original outputs for zero denominators and low-order input points.
4. In addition to per-candidate evaluation, we place the reference implementation with all optimizations disabled and the final integrated implementation with all eight optimizations enabled in one executable and directly compare them under the same board and measurement procedure.

The direct comparison reduced cycles by at least 19.52% for key generation, 19.75% for encapsulation, 11.66% for warm decapsulation with expanded-key reuse, and 15.48% for cold decapsulation including expansion from a 32-byte seed.

Artifact availability. A supplementary repository containing source code, frozen firmware, raw measurement logs, and verification scripts accompanies this submission.

2 Background and Problem Definition

2.1 Execution Structure of X-Wing

X-Wing executes ML-KEM-768 and X25519 independently and supplies both results and context information to a SHA3-256 combiner [6,13,11].

Key generation. The implementation generates an ML-KEM key pair, multiplies the fixed basepoint 9 by an X25519 secret scalar to derive a public key, and assembles the component keys in the X-Wing format. We refer to an operation whose input point is basepoint 9 as fixed-base scalar multiplication.

Encapsulation. ML-KEM produces a ciphertext and a shared secret. One ephemeral X25519 secret scalar is multiplied by basepoint 9 to produce an ephemeral public key and by the recipient's public key to produce an X25519 shared secret. We refer to the latter operation, whose input point varies with each recipient public key, as variable-base scalar multiplication. The results of both algorithms and the context information are then processed by the SHA3-256 combiner.

Decapsulation. The implementation decapsulates the ML-KEM ciphertext and selects the implicit rejection value when the ciphertext is invalid. It then computes the X25519 shared secret through variable-base scalar multiplication using the sender's public key and processes both results and the context information with the same SHA3-256 combiner. We call the condition that reuses an already expanded 2,464-byte decapsulation key *warm decapsulation*. We call the condition that includes expansion of the decapsulation key from a 32-byte seed *cold decapsulation*. The latter key expansion regenerates the X25519 public key and therefore includes fixed-base scalar multiplication.

Consequently, the fixed-base optimization applies to key generation, encapsulation, and cold decapsulation, but not to warm decapsulation. During encapsulation, the fixed-base ephemeral public key and the variable-base shared secret are computed concurrently, so the inversions required by the two outputs can share one batch inversion. We refer to this joint inversion of the two outputs as *paired inversion*.

2.2 ML-KEM Bottlenecks

ML-KEM uses a forward NTT, coefficient-wise operations, and an inverse NTT to accelerate polynomial multiplication [13]. It also repeatedly performs centered binomial distribution (CBD) sampling, serialization, compression and decompression, message conversion, and comparison. MVE arithmetic can accelerate the NTT, and independent coefficients can be processed concurrently in the data-format transformations. If the NTT output order does not match the input order required by downstream operations, however, an additional scalar reordering pass is required.

2.3 X25519 Bottlenecks

The X25519 Montgomery ladder repeatedly performs field addition, subtraction, multiplication, squaring, and a conditional swap [11]. Intermediate points remain in projective X/Z coordinates, and conversion to the final affine coordinate requires the inverse of Z. Field inversion is much more expensive than multiplication, so multiple outputs can share one inversion of the product of their denominators. A precomputation table for basepoint 9 can be used for fixed-base scalar multiplication, whereas the same table cannot be used for variable-base scalar multiplication whose input is another party's public key.

2.4 Relevant Cortex-M85 Resources

Cortex-M85 is an in-order Armv8.1-M core that provides 128-bit vector operations through MVE [3]. MVE is advantageous when the same operation is applied to several coefficients or 32-bit words. The following constraints can nevertheless limit integrated performance.

- The small scalar register file makes it difficult to keep many intermediate values live simultaneously.
- Memory accesses from MVE and scalar instructions cannot be issued concurrently without limit.
- Instruction dependencies and dual-issue rules can give different cycle counts to sequences containing the same number of instructions.
- If register pressure introduces spills or instructions that regenerate constants, their cost can exceed the benefit obtained from otherwise idle issue slots.

3 Design

3.1 Overview

Table 1 summarizes the final integrated implementation, which enables all eight optimizations. None changes the cryptographic specification or the external API.

3.2 ML-KEM: NTT Storage Order and Data-Format Transformations

After storing an NTT result, the reference ML-KEM implementation calls the scalar function `rev4` to permute groups of four coefficients into the order required by downstream operations. It also performs CBD sampling, serialization, compression, message conversion, and comparison with scalar C loops. These reordering and repeated coefficient transformations incur additional memory accesses and scalar instructions beyond the NTT arithmetic.

We connect the forward and inverse Cortex-M85 NTTs from the pqmx implementation lineage to the Plantard coefficient representation and admissible ranges used by the reference implementation [14]. We then use the MVE structured store that pqmx calls `current-order`, making the forward NTT store coefficients directly in the order required by downstream base multiplication or matrix accumulation. This removes the separate scalar reordering pass between the NTT and downstream operations. We also replace 11 data-format transformations with fixed-trip-count implementations based on MVE compiler intrinsics, extracting, quantizing, or comparing several coefficients concurrently in 128-bit vectors.

These two optimizations apply wherever ML-KEM executes: key generation, encapsulation, warm decapsulation, and cold decapsulation. Predicate management for active vector lanes and byte insertion with high data-rearrangement cost remain scalar. The optimization does not change the NTT or ML-KEM

Table 1. Eight optimizations applied to X-Wing on Cortex-M85 and the operations to which they apply.

No.	Component	Optimization	Applicable operations
1	ML-KEM	MVE NTT with downstream-compatible storage order	All
2	ML-KEM	MVE implementation of 11 data-format transformations	All
3	X25519	Width-3 signed comb for basepoint 9	Key generation, encapsulation, cold
4	X25519	Fixed stack frame and rescheduling for field multiplication	All X25519
5	X25519	Rescheduling for field squaring	All X25519
6	Common	Alignment-aware data copy and initialization	All
7	X25519	MVE xor-mask conditional swap	Encapsulation, warm, cold
8	X-Wing/X25519	Zero-safe paired inversion	Encapsulation

mathematics. It adapts the coefficient representation, range, and storage order of an existing MVE NTT to the X-Wing implementation and vectorizes only the data-format transformations whose gains remain at the complete API level.

3.3 X25519: Fixed-Base Scalar Multiplication and Field Arithmetic

The reference implementation uses the general Montgomery ladder even when the input point for public-key generation is fixed to basepoint 9. Within the ladder, `fe25519_mul` uses a push/pop frame that saves and restores registers at function entry and exit and introduces stack-pointer (SP) update dependencies. Some multiplication and squaring instructions also wait for earlier results. Repeated work in fixed-base scalar multiplication and dependency stalls in field arithmetic therefore remain distinct bottlenecks.

For fixed-base scalar multiplication, we use a width-$w = 3$ signed comb and a 33,024-byte precomputation table. Instead of secret-dependent branches or a conditional sign change of a field element, the selected sign is handled by a constant-time swap of two pointers. Intermediate values in repeated squaring chains remain in registers where possible. For field multiplication, we flatten the stack frame into fixed SP offsets and place independent instructions in dependency stalls. For field squaring, we retain the 94-instruction count while placing independent operations between instructions on long dependency chains.

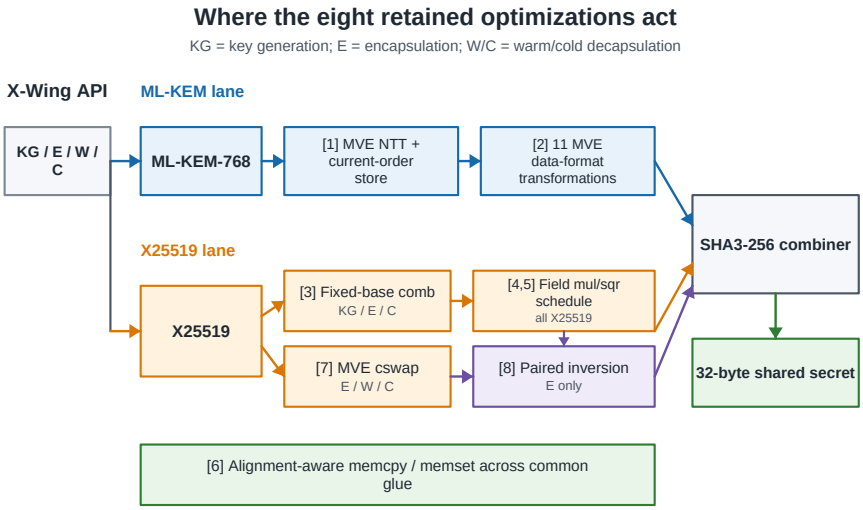

Fig. 1. X-Wing data flow and scope of the eight retained optimizations. Numbered badges refer to Table 1. The fixed-base comb is absent from warm decapsulation, MVE cswap is confined to variable-base ladders, and zero-safe paired inversion links the two X25519 outputs only during encapsulation. Alignment-aware copy and zeroing span all four API paths.

The fixed-base comb applies only to key generation, encapsulation, and cold key expansion; it does not apply to variable-base scalar multiplication or warm decapsulation, which reuses an already expanded key. In contrast, rescheduling field multiplication and squaring applies to every operation that executes X25519. Signed combs and instruction rescheduling are established techniques. Our contribution is their implementation at the exact X-Wing call sites and for the Cortex-M85 execution characteristics, together with an end-to-end measurement that includes the 33,024-byte flash cost.

3.4 X25519: MVE Conditional Swap and Paired Inversion

The reference conditional swap in the variable-base Montgomery ladder processes 16 32-bit words in a scalar xor-mask loop. During X-Wing encapsulation, one ephemeral secret scalar produces both a fixed-base ephemeral public key and a variable-base shared secret, yet the reference implementation separately inverts the denominators of the two projective-coordinate outputs. The former operation repeats the same word-level computation, while the latter duplicates an expensive field inversion.

We replace the conditional swap with four groups of 128-bit MVE load, xor, and, xor, and store operations. The instruction flow is identical whether the

mask is zero or has all bits set, and all vector temporaries are consumed within the function. This MVE conditional swap (MVE cswap) applies to encapsulation, warm decapsulation, and cold decapsulation, where the variable-base ladder executes.

Let d_1 and d_2 denote the two denominators in encapsulation. For each zero denominator, we create $\widetilde{d}_i$ by replacing the denominator with one only during the computation. The following construction reduces the two field inversions to one:

$$\begin{aligned} \widetilde{d}_i &= \begin{cases} 1, & d_i = 0, \\ d_i, & d_i \neq 0, \end{cases} \quad i \in \{1, 2\}, \\ t &= (\widetilde{d}_1 \widetilde{d}_2)^{-1}, \qquad\qquad u_1 = \widetilde{d}_2 t, \quad u_2 = \widetilde{d}_1 t. \end{aligned} \tag{1}$$

The value u_i is used for coordinate normalization and equals d_i^{-1} when $d_i \neq 0$. Zero testing compares the complete normalized field representation. A saved mask then restores the original all-zero output for a low-order input point. The tests and output masking follow the same instruction flow independently of the input. Batch inversion itself is an established technique. Our implementation contribution is the zero-safe connection of this technique at the X-Wing encapsulation point where two X25519 results coexist. Paired inversion does not apply to key generation or decapsulation, where only one output is present.

3.5 Common Memory Operations and Constant-Time Compatibility

The general-purpose `memcpy` and `memset` routines in newlib-nano support many lengths and alignments, whereas the sizes and alignments of several internal X-Wing buffers are known. We use routines that branch only on address alignment and length and perform copies and initialization one word at a time. Copies already inlined by the compiler are outside the replacement scope.

None of the eight optimizations changes the cryptographic specification or the API. Rescheduling and changes to storage order preserve fixed instruction and address patterns. The fixed-base table selection, MVE cswap, and zero-safe paired inversion use masks to avoid secret-dependent control flow. The alignment-aware memory routines branch only on public address alignment and length. Functional equivalence tests and execution-time tests with fixed and random ciphertexts support this design, but they do not constitute a leakage proof for power, electromagnetic emanations, or every cache state.

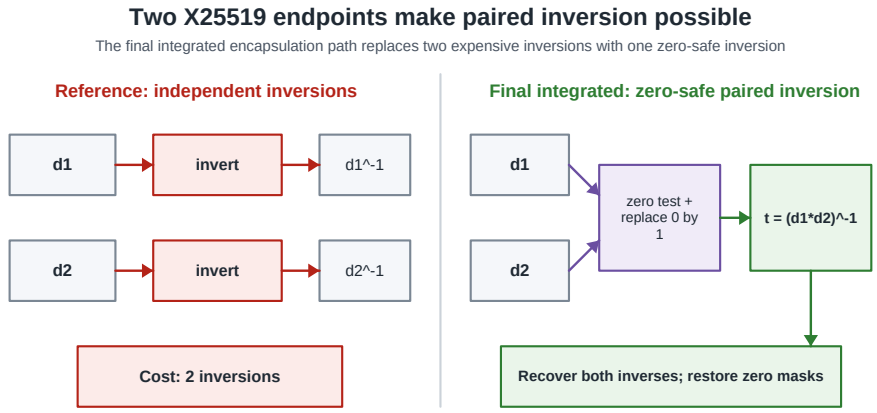

Fig. 2. Zero-safe paired inversion in X-Wing encapsulation. The reference implementation independently inverts d_1 and d_2. The final integrated implementation records a constant-time zero mask for each denominator, substitutes $\widetilde{d}_i = 1$ only during calculation when $d_i = 0$, computes $t = (\widetilde{d}_1\widetilde{d}_2)^{-1}$, and recovers $u_1 = \widetilde{d}_2 t$ and $u_2 = \widetilde{d}_1 t$. The saved masks restore the original all-zero outputs.

4 Evaluation

4.1 Reference and Final Integrated Implementations

The unoptimized reference implementation consists of ML-KEM and Keccak from the pqm4 implementation lineage, X25519 from the Lenngren implementation lineage, scalar C data-format transformations, and newlib-nano memory routines, all built for Cortex-M85. The final integrated implementation enables all eight optimizations from Sect. 3. Both implementations use the same X-Wing draft-10 specification, inputs, API, and SHA3-256 combination order.

For the primary performance comparison, the reference implementation and the final integrated implementation are included in one Executable and Linkable Format (ELF) file, and a public runtime mode selects one implementation. They therefore share the same link and execution environment, clock configuration, measurement call sites, and inputs. Experimental selection and instrumentation code is common to both executions. We consequently interpret only the difference measured within the same executable rather than treating the absolute cycle counts as the performance of a deployable library.

4.2 Device and Build

– Device: EK-RA8M1, Cortex-M85, 480 MHz
– Compiler: GCC 13.2.1, `-O2`

– Code placement: instruction tightly coupled memory (ITCM)
– Data and stack placement: data tightly coupled memory (DTCM)
– Cycle counter: Data Watchpoint and Trace CYCCNT (DWT CYCCNT)
– Each implementation-operation combination: median of 100 measurements
– Independent repetition: reflash the same board image and measure from boot

A write-stall interval observed at one specific ITCM address on the evaluation board was avoided with link padding. Before every run, we read the flash contents back and confirmed that they matched the recorded image. Only runs that passed the known-answer tests (KATs) were included in the results.

4.3 Measurement Order

The final comparison executes the two implementations in one executable in the following order:

$$\text{Reference} \rightarrow \text{Final integrated} \rightarrow \text{Final integrated} \rightarrow \text{Reference}$$

This order counterbalances effects caused by execution order. We performed five independent repetitions, reflashing the same board image before each repetition. Within each repetition, adjacent reference and final integrated executions form two pairs, yielding ten paired comparisons per operation. Let $C_{\mathrm{ref},i}(o)$ and $C_{\mathrm{final},i}(o)$ denote the cycles of the reference and final integrated implementations, respectively, for the ith paired comparison of operation o. The improvement and reported value are defined as

$$r_i(o) = 100\left(1 - \frac{C_{\mathrm{final},i}(o)}{C_{\mathrm{ref},i}(o)}\right), \qquad r_{\mathrm{reported}}(o) = \min_{1 \le i \le 10} r_i(o). \tag{2}$$

We report both the range across the ten improvements and the value defined in Eq. (2).

4.4 Measured Operations

We measure key generation, encapsulation, warm decapsulation, and cold decapsulation. Warm decapsulation reuses an expanded 2,464-byte decapsulation key. Cold decapsulation first expands the key from a compressed 32-byte seed. Because these conditions execute different work, we do not combine them into a single average for decapsulation.

4.5 Correctness Criteria

Only candidates that passed all of the following checks were included in the performance results.

– RFC 7748 X25519 KATs and SHA3-256 KATs

- ML-KEM-768 encapsulation-decapsulation round trips and implicit rejection of invalid ciphertexts
- X-Wing draft-10 Appendix C test vectors
- Byte-for-byte comparison of public keys, secret keys, ciphertexts, and shared secrets from both implementations on eight fixed inputs
- Agreement between warm and cold results and tests with three low-order X25519 input points
- Stack-boundary corruption checks and comparison of the recorded image with the actual flash contents

4.6 Candidate Selection and Contribution Analysis

Each optimization candidate is evaluated at three levels. First, we measure only the target function to establish feasibility and its local effect. Second, we connect the candidate to the actual caller and remeasure the complete key generation, encapsulation, and decapsulation APIs. Finally, we verify its effect again in a diagnostic configuration containing the retained optimizations and in the executable that compares the reference and final integrated implementations.

We quantify the individual contributions of the six-optimization diagnostic by removing one optimization at a time from the configuration containing all six. We refer to this procedure as leave-one-out analysis. Intermediate and final experiments can use different executables and measurement call sites; therefore, we neither add nor multiply improvements obtained from different experiments.

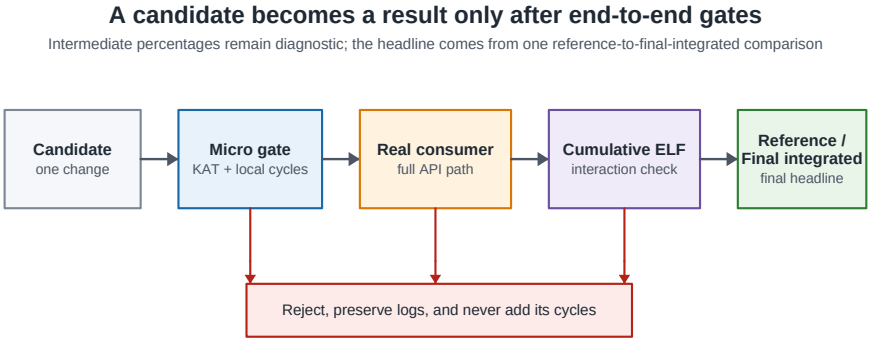

Fig. 3. Candidate adoption and reporting procedure. Each change first passes correctness and function-level measurement, is then integrated into a real X-Wing call path, and is remeasured at API level. Retained changes are checked in a combined diagnostic build. Intermediate and leave-one-out percentages remain diagnostic; the reported result is the minimum of ten paired improvements from the final same-executable comparison between the reference and final integrated implementations.

5 Results

5.1 Final Performance

Table 2 presents the final end-to-end results.

These results are the final effects of applying all eight optimizations to the complete X-Wing API. The subsequent tables are diagnostic evidence explaining where the final effects arise; they are not quantities to be added to the final results. All five independent repetitions used the same ELF file and the same S-record (SREC) board image. We verified the flash contents after every programming step, and every run passed all functional and stack-boundary checks. The narrow improvement ranges show that the difference between the reference and final integrated implementations is reproducible under the same board and measurement conditions.

5.2 Integrated Effect

We first performed a separate diagnostic experiment that combined six of the final eight optimizations: ML-KEM NTT and storage order, ML-KEM data-format transformations, the fixed-base comb, field multiplication, field squaring, and alignment-aware memory routines. This experiment also placed the configuration with all six disabled and the configuration with all six enabled in one executable for a direct comparison. Key generation, encapsulation, warm decapsulation, and cold decapsulation improved by 25.96%, 20.25%, 11.22%, and 18.30%, respectively.

Table 2. End-to-end cycles of the reference and final integrated implementations alternated within one executable. Each cycle count is the median of 100 measurements. The ranges summarize ten paired comparisons from five independent reflashes. The reported value is the minimum observed improvement defined in Eq. (2).

Operation	Reference cycles	Integrated cycles	Improvement range	Reported
Key generation	870,869–870,970	700,834–700,865	19.521–19.534%	**19.52%**
Encapsulation	1,326,924–1,326,931	1,064,853–1,064,862	19.750–19.751%	**19.75%**
Warm decapsulation	934,803–934,837	825,746–825,793	11.661–11.670%	**11.66%**
Cold decapsulation	1,802,626–1,802,655	1,523,463–1,523,502	15.484–15.488%	**15.48%**

These values are direct comparisons between configurations before and after all six optimizations; they are not sums of six function-level improvements. The six-optimization diagnostic and the final direct comparison use different executables and common measurement code. We therefore do not add the improvements of the two additional optimizations to values such as 25.96% to construct the final improvement.

5.3 Individual Contributions

Table 3 reports the cycle increases observed when each optimization was removed individually from the six-optimization diagnostic.

The fixed-base comb makes the largest individual contribution to key generation, encapsulation, and cold decapsulation, while its effect on warm decapsulation is at the noise level. This result agrees with the applicability described in Sect. 2. The NTT and data-format transformations contribute to all four measured operations in which ML-KEM executes, although their magnitudes differ because the call structures differ.

The sum of the six contributions computed by leave-one-out analysis accounts for 91.4–99.999% of the total before-after difference observed within the same experiment. We attribute the remaining difference to interactions among optimizations rather than assigning it arbitrarily to one optimization.

Table 3. Leave-one-out results for the six-optimization diagnostic. Each value is the cycle increase caused by removing one optimization and must not be added to the improvements from the final direct comparison.

Optimization	Key generation	Encapsulation	Warm	Cold
Fixed-base comb	152,860	152,710	16	152,732
NTT and storage order	22,556	18,002	28,180	50,808
MVE data-format transformations	11,120	33,300	39,386	50,552
Alignment-aware memory routines	12,660	10,042	9,906	21,380
Field-squaring rescheduling	3,488	22,770	19,296	22,794
Field-multiplication rescheduling	4,182	11,950	7,632	11,898

5.4 Additional Techniques

Starting from the six-optimization diagnostic used in Sect. 5.2, we enabled the MVE cswap and paired inversion separately and together. The values in Table 4 come from this separate diagnostic and are not added to the final direct comparison.

Encapsulation with both optimizations is faster than encapsulation with paired inversion alone, so the two optimizations do not eliminate exactly the same cost. Paired inversion applies only to encapsulation, where two outputs coexist; key generation contains no applicable site for either optimization.

A separate candidate reordered the MVE cswap instructions to account for the beat-level execution characteristics of MVE and reduced the function cost from 62 cycles to 54 cycles. Its end-to-end improvement for cold decapsulation was only 0.162–0.163%, below the predefined 0.20% threshold, so this candidate was excluded from the final integrated implementation.

Table 4. End-to-end improvements obtained by adding MVE cswap and paired inversion to the six-optimization diagnostic.

Candidate	Applicable operation	Complete-API improvement
MVE cswap	Encapsulation	1.264–1.265%
Paired inversion	Encapsulation	2.957%
Both techniques	Encapsulation	**4.226–4.228%**
MVE cswap	Warm decapsulation	1.707–1.709%
MVE cswap	Cold decapsulation	0.928%

5.5 Resource Cost

We measured time and space in separate builds of the six-optimization diagnostic after removing the experimental implementation selector and instrumentation to approximate a deployable library. Table 5 presents the results.

Table 5. Time-space cost of a separate six-optimization build approximating a deployable library. The memory increase in this table is not the exact size of the complete final integrated implementation.

Measurement	Cost or effect
Nonvolatile-memory increase	34,308 bytes
Static-RAM increase	2,040 bytes
Fixed-base comb precomputation table	33,024 bytes, 96.25% of the nonvolatile-memory increase
Removing the precomputation table	Approximately 152,000 additional cycles in key generation, encapsulation, and cold decapsulation

The fixed-base comb is therefore not a cost-free performance improvement; it is a tradeoff between flash usage and execution time. Products with limited memory can consider a configuration without the comb as a separate speed-memory option.

6 Limitations

6.1 Generalizability

The primary experiments used one EK-RA8M1 board. Reflashing the same image five times confirmed software-level repeatability, but we did not measure a second board without the anomalous ITCM interval, another Cortex-M85 system-on-chip, another compiler, or another clock configuration.

The reference implementation is not an external reference point that guarantees the highest performance among all publicly available Cortex-M85 X-Wing

implementations. The comparison between the reference and final integrated implementations quantifies the combined effect of the eight optimizations within the same implementation lineage. We do not construct performance rankings from external cycle counts obtained with different devices, APIs, or protocol versions.

6.2 Resource Measurement

We measured the flash increase caused by the fixed-base table and checked for stack-boundary corruption. An exact measurement of the code and data size of the final integrated library still requires a build that removes all experimental selector and instrumentation code. We did not measure power or energy and therefore do not claim that a cycle reduction implies an equal percentage reduction in energy.

6.3 Security Validation

The implementations passed KATs, byte-for-byte equivalence checks, invalid- ciphertext rejection tests, and low-order input-point tests. In Welch's t-test comparing execution times for fixed and random ciphertexts, all 24 evaluated t values had absolute values below 2.0, and the test did not detect an execution-time difference between the two input groups. This result does not replace a formal constant-time proof or a power or electromagnetic side-channel evaluation.

7 Conclusion

This work goes beyond porting X-Wing components individually to Cortex-M85. It jointly optimizes ML-KEM data parallelism and the fixed-base scalar multiplication, field arithmetic, conditional swap, and inversion structure of X25519 across the complete protocol. Direct comparison of the reference and final integrated implementations in one executable showed cycle reductions of 19.52% for key generation, 19.75% for encapsulation, 11.66% for warm decapsulation, and 15.48% for cold decapsulation.

The largest individual contribution was the fixed-base comb with its 33,024-byte precomputation table. MVE implementations of the ML-KEM NTT storage order and data-format transformations, X25519 field-arithmetic rescheduling, and common memory optimization reduced the remaining cost. The final integrated implementation adds MVE cswap and zero-safe paired inversion to the six-optimization configuration.

These results show that embedded cryptographic optimization requires attention not only to faster individual functions but also to where an optimization applies, its additional resource cost, and remeasurement at the complete API level. Future work will remeasure end-to-end performance on other Cortex-M85 systems-on-chip, other Arm M-profile cores, and different compiler and memory configurations to assess reproducibility and architecture dependence. It will also

measure the exact flash and RAM usage of a deployable build without instrumentation.

References

1. Abdulrahman, A., Becker, H., Kannwischer, M.J., Klein, F.: Fast and clean: Auditable high-performance assembly via constraint solving. IACR Transactions on Cryptographic Hardware and Embedded Systems **2024**(1), 87–132 (2023). https://doi.org/10.46586/tches.v2024.i1.87-132
2. Abdulrahman, A., Kannwischer, M.J., Lim, T.H.: Enabling microarchitectural agility: Taking ML-KEM and ML-DSA from Cortex-M4 to M7 with SLOTHY. In: Proceedings of the 20th ACM Asia Conference on Computer and Communications Security. pp. 1756–1771. ACM (2025). https://doi.org/10.1145/3708821.3736210
3. Arm Limited: Arm Cortex-M85 processor software optimization guide. Tech. Rep. 107950_0100_01_en, Arm Limited (2023), issue 01
4. Barbosa, M., Connolly, D., Duarte, J.D., Kaiser, A., Schwabe, P., Varner, K., Westerbaan, B.: X-Wing: The hybrid KEM you've been looking for. IACR Communications in Cryptology **1**(1) (2024). https://doi.org/10.62056/a3qj89n4e
5. Becker, H., Bermudo Mera, J.M., Karmakar, A., Yiu, J., Verbauwhede, I.: Polynomial multiplication on embedded vector architectures. IACR Transactions on Cryptographic Hardware and Embedded Systems **2022**(1), 482–505 (2021). https://doi.org/10.46586/tches.v2022.i1.482-505
6. Connolly, D., Schwabe, P., Westerbaan, B.E.: X-Wing: General-purpose hybrid post-quantum KEM. Internet-Draft draft-connolly-cfrg-xwing-kem-10 (2026), work in Progress, published 2 March 2026; expires 3 September 2026
7. Düll, M., Haase, B., Hinterwälder, G., Hutter, M., Paar, C., Sánchez, A.H., Schwabe, P.: High-speed Curve25519 on 8-bit, 16-bit, and 32-bit microcontrollers. Designs, Codes and Cryptography **77**(2–3), 493–514 (2015). https://doi.org/10.1007/s10623-015-0087-1
8. Fujii, H., Aranha, D.F.: Curve25519 for the Cortex-M4 and beyond. In: Progress in Cryptology – LATINCRYPT 2017. LNCS, vol. 11368, pp. 109–127. Springer (2019). https://doi.org/10.1007/978-3-030-25283-0_6
9. Hankerson, D., Menezes, A.J., Vanstone, S.: Guide to Elliptic Curve Cryptography. Springer (2004). https://doi.org/10.1007/b97644
10. Kannwischer, M.J., Rijneveld, J., Schwabe, P., Stoffelen, K.: pqm4: Testing and benchmarking NIST PQC on ARM Cortex-M4. IACR ePrint 2019/844 (2019), source snapshot: GitHub commit cc2c1b992b60 (19 June 2026)
11. Langley, A., Hamburg, M., Turner, S.: Elliptic curves for security. Tech. Rep. RFC 7748, IETF (2016)
12. Lenngren, E.: X25519 for ARM Cortex-M4. GitHub repository Emill/X25519-Cortex-M4 (2020), source snapshot, 4 February 2020
13. NIST: Module-lattice-based key-encapsulation mechanism standard. Tech. Rep. FIPS 203, NIST (2024)
14. slothy-optimizer: pqmx: Post-quantum cryptography on Arm Cortex-M. GitHub repository slothy-optimizer/pqmx (2026), commit 89465b9756cd (15 June 2026)